

Introducción a Javascript

Índice

1 - Tema 1 - Introducción y programación básica.....	6
1.1 ¿Qué es JavaScript?.....	6
1.2 Cómo incluir JavaScript en documentos HTML.....	6
1.2.1 En el mismo documento.....	6
1.2.2 En un archivo externo (recomendado).....	7
1.2.3 Dentro de elementos del HTML.....	7
1.3 Ubicación de las etiquetas script.....	8
1.4 La etiqueta noscript.....	8
1.5 Detectar errores en JavaScript.....	8
1.5.1 Consola y mensajes de error.....	9
1.6 Sintaxis.....	9
1.7 El primer script.....	10
1.7.1 Ejercicio 1.1.....	11
1.8 Programación básica.....	11
1.8.1 Comunicación con el usuario.....	11
1.8.2 Variables.....	12
1.8.3 Tipos de variables.....	13
Ejercicio 1.2.....	15
1.8.4 Booleanos.....	15
1.8.5 Conversiones entre tipos simples.....	16
Ejercicio 1.3.....	18
1.8.6 Operadores.....	18
1.9 Estructuras de control de flujo.....	23

1.9.1 Estructura if.....	23
1.9.2 Estructura if...else.....	24
1.9.3 Estructura switch.....	24
1.9.4 Estructura for.....	25
1.9.5 Estructura While.....	26
1.9.6 estructura do...while.....	26
1.9.7 Sentencias break y continue.....	27
Ejercicio 1.4.....	28
Ejercicio 1.5.....	28
Ejercicio 1.6.....	29



1 - Tema 1 - Introducción y programación básica

1.1 ¿Qué es JavaScript?

JavaScript es un lenguaje de programación que se utiliza principalmente para crear páginas web dinámicas.

Una página web dinámica es aquella que incorpora efectos como texto que aparece y desaparece, animaciones, acciones que se activan al pulsar botones y ventanas con mensajes de aviso al usuario.

Técnicamente, JavaScript es un lenguaje de programación interpretado, por lo que no es necesario compilar los programas para ejecutarlos. En otras palabras, los programas escritos con JavaScript se pueden probar directamente en cualquier navegador sin necesidad de procesos intermedios.

A pesar de su nombre, JavaScript no guarda ninguna relación directa con el lenguaje de programación Java. Legalmente, JavaScript es una marca registrada de la empresa Sun Microsystems, como se puede ver en <http://www.sun.com/suntrademarks/>.

1.2 Cómo incluir JavaScript en documentos HTML

1.2.1 En el mismo documento

El código JavaScript se encierra entre etiquetas `<script>` y se incluye en cualquier parte del documento. Aunque es correcto incluir cualquier bloque de código en cualquier zona de la página, se recomienda definir el código JavaScript dentro de la cabecera del documento (dentro de la etiqueta `<head>`):

```
<!DOCTYPE html >
<html lang="es">
  <head>
    <meta charset="UTF-8">
    <title>Ejemplo de código JavaScript en el propio documento</title>
    <script>
      console.log("Hola mundo");
    </script>
  </head>
  <body>
    <p>Un párrafo de texto.</p>
  </body>
</html>
```



1.2.2 En un archivo externo (recomendado)

Archivo codigo.js `alert("Un mensaje de prueba");`

Documento HTML

```
<!DOCTYPE html>

<html lang="es">
  <head>
    <meta charset="UTF-8">
    <title>Ejemplo de código JavaScript en el propio documento</title>
    <script src="/js/codigo.js"></script>
  </head>
  <body>
    <p>Un párrafo de texto.</p>
  </body>
</html>
```

Los archivos de tipo JavaScript son documentos normales de texto con la extensión .js, que se pueden crear con cualquier editor de texto como Visual Studio Code (el que nosotros utilizaremos), Notepad++, Vi, etc.

La principal ventaja de enlazar un archivo JavaScript externo es que se simplifica el código HTML de la página y que se puede reutilizar el mismo código JavaScript en todas las páginas del sitio web

1.2.3 Dentro de elementos del HTML

Este último método es el menos utilizado, ya que consiste en incluir trozos de JavaScript dentro del código HTML de la página:

```
<!DOCTYPE html>
<html>
  <head>
    <meta charset="UTF-8">
    <title>Ejemplo de código JavaScript en el propio documento</title>
  </head>
  <body>
    <p onclick="alert('Un mensaje de prueba')">Un párrafo de texto.</p>
  </body>
</html>
```

El mayor inconveniente de este método es que *ensucia* innecesariamente el código HTML de la página y complica el mantenimiento del código JavaScript. En general, este método sólo se utiliza para definir algunos eventos y en algunos otros casos especiales, como se verá más adelante.



1.3 Ubicación de las etiquetas script

Si utilizamos una etiqueta script para incluir el código JavaScript en nuestras páginas (tanto incorporado en la página como desde un fichero externo), conviene tener presente cuál es la mejor ubicación de dicho fichero.

- **Si el fichero sólo contiene funciones** que se van a activar o lanzar más adelante, a medida que interactuemos con la página, **podemos colocar este script en la cabecera (head).**
- **Si el fichero contiene instrucciones que necesitan ejecutarse inmediatamente** cuando la página se cargue, **entonces conviene incorporarlo al final de la página**, cuando ya se haya cargado todo el contenido de la misma:

1.4 La etiqueta noscript

Algunos navegadores no disponen de soporte completo de JavaScript, otros navegadores permiten bloquearlo parcialmente e incluso algunos usuarios bloquean completamente el uso de JavaScript porque creen que así navegan de forma más segura.

El lenguaje HTML define la etiqueta `<noscript>` para mostrar un mensaje al usuario cuando su navegador no puede ejecutar JavaScript. El siguiente código muestra un ejemplo del uso de la etiqueta `<noscript>`:

```
<head> ... </head>
<body>
  <noscript>
    <p>Bienvenido a Mi Sitio</p>
    <p>La página que estás viendo requiere para su funcionamiento el uso de JavaS-
cript.
    Si lo has deshabilitado intencionadamente, por favor vuelve a activarlo.</p>
  </noscript>
</body>
```

La etiqueta `<noscript>` se debe incluir en el interior de la etiqueta `<body>` (normalmente se incluye al principio de `<body>`). El mensaje que muestra `<noscript>` puede incluir cualquier elemento o etiqueta HTML.

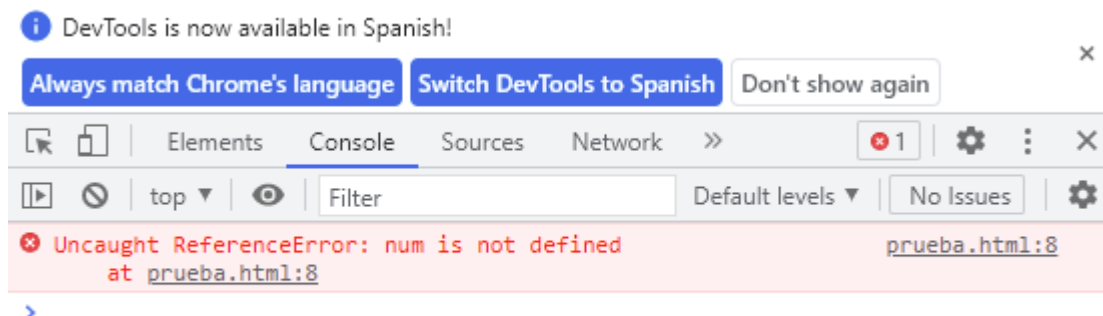
1.5 Detectar errores en JavaScript

A diferencia de los lenguajes de programación de aplicaciones de escritorio, como Java o C#, en lenguajes de programación web como JavaScript o PHP es más difícil ver dónde está el error cuando una aplicación o página web no hace lo que debe, ya que no existe un terminal o consola accesible donde poder ver lo que el programa está haciendo.

La mayoría de navegadores proporcionan herramientas para poder depurar nuestro código JavaScript, y ver dónde están los errores cuando cargamos las páginas. Además, disponen de herramientas para revisar el código HTML y CSS y corregir posibles errores de diseño o estructura.



En Google Chrome, por ejemplo, con la tecla F12 (o haciendo clic derecho en la página y eligiendo la opción Inspeccionar) mostramos el panel de herramientas para desarrolladores. En él encontraremos varias secciones o pestañas. Una de las más útiles son la Consola, que muestra posibles errores en la ejecución del código JavaScript.



1.5.1 Consola y mensajes de error

Además de la instrucción `alert`, con la que podemos mostrar al usuario mensajes en la página a modo de popups, también podemos emplear la instrucción **`console.log`** o **`console.error`** para mostrar mensajes directamente en esta consola de depuración. Esto nos puede venir bien para dejar un registro de mensajes durante la ejecución del programa, y también para informar de errores que se puedan producir y que hayamos detectado.

```
console.error("El email está vacío");
```

1.6 Sintaxis

La sintaxis de un lenguaje de programación se define como el conjunto de reglas que deben seguirse al escribir el código fuente de los programas para considerarse como correctos para ese lenguaje de programación.

La sintaxis de JavaScript es muy similar a la de otros lenguajes de programación como Java y C.

Las normas básicas que definen la sintaxis de JavaScript son las siguientes:

- **No se tienen en cuenta los espacios en blanco y las nuevas líneas:** como sucede con HTML, el intérprete de JavaScript ignora cualquier espacio en blanco sobrante, por lo que el código se puede ordenar de forma adecuada para entenderlo mejor (tabulando las líneas, añadiendo espacios, creando nuevas líneas, etc.)
- **Se distinguen las mayúsculas y minúsculas:**
- **No se define el tipo de las variables:** al crear una variable, no es necesario indicar el tipo de dato que almacenará. De esta forma, una misma variable puede almacenar diferentes tipos de datos durante la ejecución del script.
- **No es necesario terminar cada sentencia con el carácter de punto y coma (;):** en la mayoría de lenguajes de programación, es obligatorio terminar cada sentencia con el carácter ;. Aunque



JavaScript no obliga a hacerlo, es conveniente seguir la tradición de terminar cada sentencia con el carácter del punto y coma (;).

- **Se pueden incluir comentarios:** los comentarios se utilizan para añadir información en el código fuente del programa. JavaScript define dos tipos de comentarios: los de una sola línea y los que ocupan varias líneas.

Ejemplo de comentario de una sola línea:

```
// a continuación se muestra un mensaje  
  
alert("mensaje de prueba");
```

Los comentarios de una sola línea se definen añadiendo dos barras oblicuas (//) al principio de la línea.

Ejemplo de comentario de varias líneas:

```
/* Los comentarios de varias líneas son muy útiles cuando se necesita incluir bastante información  
en los comentarios */  
  
alert("mensaje de prueba");
```

Los comentarios multilínea se definen encerrando el texto del comentario entre los símbolos /* y */.

1.7 El primer script

A continuación, se muestra un primer script sencillo pero completo:

```
<!DOCTYPE html>  
<html >  
  <head>  
    <meta charset="UTF-8">  
    <title>El primer script</title>  
    <script type="text/javascript">  
      alert("Hola Mundo!");  
    </script>  
  </head>  
  <body>  
    <p>Esta página contiene el primer script</p>  
  </body>  
</html>
```

La instrucción alert() es una de las utilidades que incluye JavaScript y permite mostrar un mensaje en la pantalla del usuario. Si se visualiza la página web de este primer script en cualquier navegador, automáticamente se mostrará una ventana con el mensaje "Hola Mundo!".



1.7.1 Ejercicio 1.1

Modificar el primer script para que:

1. Todo el código JavaScript se encuentre en un archivo externo llamado `codigo.js` y el script siga funcionando de la misma manera.
2. Después del primer mensaje, se debe mostrar otro mensaje que diga *"Soy el primer script"*
3. Añadir algunos comentarios que expliquen el funcionamiento del código
4. Añadir en la página HTML un mensaje de aviso para los navegadores que no tengan activado el soporte de JavaScript

1.8 Programación básica

Antes de comenzar a desarrollar programas y utilidades con JavaScript, es necesario conocer los elementos básicos con los que se construyen las aplicaciones. Si ya sabes programar en algún lenguaje de programación, este capítulo te servirá para conocer la sintaxis específica de JavaScript.

Si nunca has programado, este capítulo explica en detalle y comenzando desde cero los conocimientos básicos necesarios para poder entender posteriormente la programación avanzada, que es la que se utiliza para crear las aplicaciones reales.

1.8.1 Comunicación con el usuario

Ya hemos visto que con la instrucción ***alert*** podemos sacar mensajes por pantalla en un pequeño cuadro de diálogo, poniendo entre paréntesis el mensaje que queremos mostrar. Con las instrucciones `console.log` y `console.error` estos mensajes se muestran en la consola del navegador, que no está directamente visible o accesible por los usuarios:

```
alert("Hola, buenos días");  
  
console.log("El email está vacío");
```

Alternativamente, también podemos emplear la instrucción ***document.write***, que muestra la información directamente en la página HTML donde se ejecuta el código JavaScript:

```
document.write("Esto es un texto en el HTML");
```




Otra instrucción útil para comunicarnos de forma básica con el usuario es `prompt`. Al igual que en el caso anterior, entre paréntesis podemos mostrar un mensaje, pero con esta instrucción también podemos permitir que el usuario escriba algo (en un cuadro de texto), recoger lo que ha escrito y guardarlo en una variable, para luego poderlo procesar y utilizar.

```
let nombre = prompt("Dime tu nombre");
```

Sacaría un cuadro para que el usuario escribiese su nombre, y al pulsar en Aceptar quedaría guardado en la variable `nombre`. Si finalmente el usuario cancela, la variable queda con un valor nulo (`null`).

1.8.2 Variables

Las variables en JavaScript se pueden crear de tres formas:

<code>let n1 = 5;</code>	Aconsejado , crea una variable local al bloque de código, (más adelante se verán las diferencias entre variables locales y globales)
<code>var n2 = 5;</code>	Desaconsejado , crea una variable global
<code>n3 = 5;</code>	Desaconsejado , crea una variable global

La palabra reservada **let** solamente se debe indicar al definir por primera vez la variable, lo que se denomina **declarar** una variable.

Si cuando se declara una variable se le asigna también un valor, se dice que la variable ha sido **inicializada**. En JavaScript no es obligatorio inicializar las variables, ya que se pueden declarar por una parte y asignarles un valor posteriormente.:

```
let numero_1;

let numero_2;

numero_1 = 3;

numero_2 = 1;

let resultado = numero_1 + numero_2;
```

El nombre de una variable también se conoce como **identificador** y debe cumplir las siguientes normas:

- Sólo puede estar formado por letras, números y los símbolos \$ (dólar) y _ (guion bajo).
- El primer carácter no puede ser un número.

Por tanto, las siguientes variables tienen nombres correctos:



```
let $numero1;  
  
let _$letra;  
  
let $$$otroNumero;  
  
let $_a_$4
```

Sin embargo, las siguientes variables tienen identificadores incorrectos:

```
let 1numero; // Empieza por un número  
  
let numero;1_123; // Contiene un carácter";"
```

1.8.3 Tipos de variables

1.8.3.1 Numéricas

Se utilizan para almacenar valores numéricos enteros o decimales. En este caso, el valor se asigna indicando directamente el número entero o decimal. Los números decimales utilizan el carácter . (punto) en vez de , (coma) para separar la parte entera y la parte decimal:

```
let iva = 16; // variable tipo entero  
  
let total = 234.65; // variable tipo decimal
```

1.8.3.2 Cadenas de texto o strings

Se utilizan para almacenar caracteres, palabras y/o frases de texto. Para asignar el valor a la variable, se encierra el valor entre comillas dobles o simples, para delimitar su comienzo y su final:

```
let mensaje = "Bienvenido a nuestro sitio web";  
  
let nombreProducto = 'Producto ABC';  
  
let letraSeleccionada = 'c';
```



En ocasiones, el texto que se almacena en las variables no es tan sencillo. Si por ejemplo el propio texto contiene comillas simples o dobles, la estrategia que se sigue es la de encerrar el texto con las comillas (simples o dobles) que no utilice el texto:

```
let texto1 = "Una frase con 'comillas simples' dentro";
```

```
let texto2 = 'Una frase con "comillas dobles" dentro';
```

No obstante, a veces las cadenas de texto contienen tanto comillas simples como dobles. Además, existen otros caracteres que son difíciles de incluir en una variable de texto (tabulador, ENTER, etc.) Para resolver estos problemas, JavaScript define un mecanismo que consiste en sustituir el carácter problemático por una combinación simple de caracteres. A continuación se muestra la tabla de conversión que se debe utilizar:

Si se quiere incluir...	Se debe incluir...
Una nueva línea	\n
Un tabulador	\t
Una comilla simple	\'
Una comilla doble	\"
Una barra inclinada	\\

De esta forma, el ejemplo anterior que contenía comillas simples y dobles dentro del texto se puede rehacer de la siguiente forma:

```
let texto1 = 'Una frase con \'comillas simples\' dentro';
```

```
let texto2 = "Una frase con \"comillas dobles\" dentro";
```

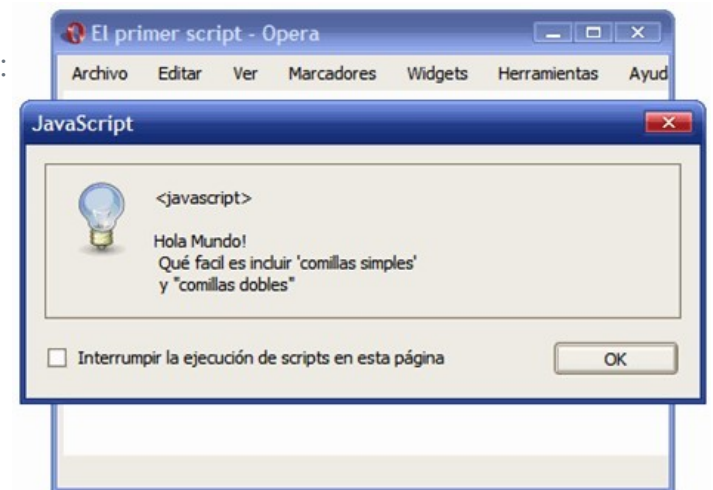
Este mecanismo de JavaScript se denomina *"mecanismo de escape"* de los caracteres problemáticos, y es habitual referirse a que los caracteres han sido *"escapados"*.



Ejercicio 1.2

Crea un documento llamado **02_comillas_js.html** modificando el ejercicio anterior para que:

1. El mensaje que se muestra al usuario se almacene en una variable llamada `mensaje` y el funcionamiento del script sea el mismo.
2. El mensaje mostrado sea el de la siguiente imagen:



1.8.4 Booleanos

Las variables de tipo *boolean* o *booleano* también se conocen con el nombre de variables de tipo lógico.

Una variable de tipo *boolean* almacena un tipo especial de valor que solamente puede tomar dos valores: `true` (verdadero) o `false` (falso). No se puede utilizar para almacenar números y tampoco permite guardar cadenas de texto.

```
let clienteRegistrado = false;  
  
let ivaIncluido = true;
```



1.8.5 Conversiones entre tipos simples

En ocasiones nos puede interesar convertir un tipo simple en otro. Por ejemplo, convertir un texto a entero, o un número en texto. Para ello JavaScript dispone de distintas instrucciones de conversión.

Por ejemplo, la instrucción `Number` permite convertir un valor a algo numérico. Podemos emplearlo para convertir textos a enteros o reales:

```
let texto = "32";  
let numero = Number(texto); // 32
```

Alternativamente, también disponemos de las instrucciones `parseInt` y `parseFloat` para convertir a entero o real, respectivamente.

```
let texto = "32";  
let numero = parseInt(texto); // 32
```

Para hacer el paso opuesto (convertir un número, o cualquier otro dato, a texto), podemos emplear la instrucción `String`:

```
let numero = 23;  
let texto = String(numero); // "23"
```

Del mismo modo, también podemos emplear la instrucción `Boolean` para convertir un dato a booleano.

```
let texto = "true";  
let resultado = Boolean(texto); // true
```

Una de las aplicaciones prácticas que tiene esto es poder convertir datos recogidos del usuario a otros tipos. Por ejemplo, ya hemos visto que con la instrucción `prompt` podemos pedirle al usuario que introduzca algún dato. Estos datos, JavaScript los interpreta directa-



mente como texto. Así, si el usuario escribe "32", JavaScript no lo va a interpretar como que ha escrito el número 32, sino como que ha escrito los símbolos 3 y 2, simplemente.

Si queremos tratar lo que ha escrito el usuario como un número, para luego poder hacer operaciones con ese dato (sumarlo, por ejemplo), deberemos convertir el dato a numérico. El siguiente ejemplo calcula el doble del número que indique el usuario, usando `parseInt`.

```
let numero = prompt("Dime un número");
let doble = parseInt(numero) * 2;
alert("El doble es " + doble);
```

NaN

En algunas conversiones u operaciones debemos tener especial cuidado con que los datos con que trabajamos sean numéricos, ya que de lo contrario el resultado que se va a producir va a ser NaN (abreviatura de Not a Number). Por ejemplo, si sumamos un entero y un valor `undefined`, el resultado será NaN.

```
let numero;
let otroNumero = parseInt(numero);
// otroNumero es NaN porque numero es *undefined*
```

Disponemos de la instrucción `isNaN` para comprobar si un dato no es numérico (devuelve `true` cuando no lo es):

```
if (isNaN(valor))
{
    alert("El valor no es numérico");
}
```



Ejercicio 1.3

Crea un documento llamado **03_Operaciones_js.html** y, con JavaScript, pídele al usuario que introduzca dos números (primero uno y luego otro) y saca cuatro mensajes mostrando su suma, resta, multiplicación y división. (como hay una división lo mejor es pasar los datos leídos a números decimales o reales con `parseFloat`)

Ejemplo:
Introduce el primer número: 6
Introduce el segundo número: 4
suma: 10
resta: 2
multiplicación: 24
división: 1,5

1.8.6 Operadores

Los operadores permiten manipular el valor de las variables, realizar operaciones matemáticas con sus valores y comparar diferentes variables. De esta forma, los operadores permiten a los programas realizar cálculos complejos y tomar decisiones lógicas en función de comparaciones y otros tipos de condiciones.

1.8.6.1 Asignación

Este operador se utiliza para guardar un valor específico en una variable. El símbolo utilizado es `=` (no confundir con el operador `==` que se utiliza para comparar y se verá más adelante)

```
let numero1 = 3;
```

A la izquierda del operador, siempre debe indicarse el nombre de una variable. A la derecha del operador, se pueden indicar variables, valores, condiciones lógicas, etc:

```
let numero1 = 3;  
  
let numero2 = 4;  
  
// Ahora, la variable numero1 vale 5  
numero1 = 5;  
  
// Ahora, la variable numero1 vale 4  
numero1 = numero2;
```



1.8.6.2 Incremento y decremento ++ --

Estos dos operadores solamente son válidos para las variables numéricas y se utilizan para incrementar o decrementar en una unidad el valor de una variable y pueden ir delante o detrás de la variable con diferente resultado

```
let a = 1;

let b = 5;

console.log(a++); // Imprime 1 y incrementa a (2)

console.log(++a); // Incrementa a (3), e imprime 3
```

1.8.6.3 Lógicos

El resultado de cualquier operación que utilice operadores lógicos siempre es un valor lógico o *booleano*.

1.8.6.4 Negación !

Uno de los operadores lógicos más utilizados es el de la negación. Se utiliza para obtener el valor contrario al valor de la variable:

```
let visible = true;
alert(!visible); // Muestra "false" y no "true"
```

La negación lógica se obtiene prefijando el símbolo ! al identificador de la variable. El funcionamiento de este operador se resume en la siguiente tabla:

variable	!variable
true	false
false	true

1.8.6.5 AND &&

La operación lógica AND obtiene su resultado combinando dos valores booleanos. El operador se indica mediante el símbolo && y su resultado solamente es true si los dos operandos son true:



variable1	variable2	variable1 && variable2
true	true	true
true	false	false
false	true	false
false	false	false

```
var valor1 = true;
var valor2 = false;
resultado = valor1 && valor2; // resultado = false

valor1 = true;
valor2 = true;
resultado = valor1 && valor2; // resultado = true
```

1.8.6.6 OR ||

La operación lógica OR también combina dos valores booleanos. El operador se indica mediante el símbolo || y su resultado es true si alguno de los dos operandos es true:

variable1	variable2	variable1 variable2
true	true	true
true	false	true
false	true	true
false	false	false

```
var valor1 = true;
var valor2 = false;
resultado = valor1 || valor2; // resultado = true

valor1 = false;
valor2 = false;
resultado = valor1 || valor2; // resultado = false
```

1.8.6.7 Aritméticos

- Los operadores aritméticos son: resta (-), multiplicación (*), división (/) y módulo (%).
- Estos operadores operan siempre con números, por tanto, cualquier operando que no sea un número debe ser convertido a número.

```
console.log(4 * 6); // Imprime 24

console.log("Hello " * "world!"); // Imprime NaN

console.log("24" / 12); // Imprime 2 (24 / 12)

console.log("42" * true); // Imprime 42 (42 * 1)

console.log("42" * false); // Imprime 0 (42 * 0)
```



```
console.log("42" * undefined); // Imprime NaN

console.log("42" - null); // Imprime 42 (42 - 0)

console.log(12 * "hello"); //Imprime NaN("hello" no puede ser convertido a
número)

console.log(13 * 10 - "12"); // Imprime 118 ((13 * 10) - 12)
```

- el operador "módulo", calcula el resto de la división entera de dos números. Si se divide por ejemplo 10 y 5, la división es exacta y da un resultado de 2. Sin embargo, si se divide 9 y 5, la división no es exacta, el resultado es 1 y el resto 4, por lo que módulo de 9 y 5 es igual a 4.

```
let numero1 = 10; let numero2 = 5;

let resultado = numero1 % numero2; // resultado = 0

numero1 = 9; numero2 = 5;

resultado = numero1 % numero2; // resultado = 4
```

Los operadores matemáticos también se pueden combinar con el operador de asignación para abreviar su notación:

```
let numero1 = 5;

numero1 += 3; // numero1 = numero1 + 3 => 8

numero1 -= 1; // numero1 = numero1 - 1 => 4

numero1 *= 2; // numero1 = numero1 * 2 => 10

numero1 /= 5; // numero1 = numero1 / 5 => 1

numero1 %= 4; // numero1 = numero1 % 4 => 1
```

1.8.6.8 Relacionales

- Estos operadores son: >, <, <=, >=, ==, ===, !=, !==
- Los operadores de comparación, comparan dos valores y devuelven un booleano (true o false)
- Estos operadores son prácticamente los mismos que en la mayoría de lenguajes de programación, a excepción de algunos, que veremos a continuación.



- Podemos usar `==` o `===` para comparar la igualdad (o lo contrario `!=`, `!==`).
 - La principal diferencia es que el primero, no tiene en cuenta los tipos de datos que están siendo comparados, compara si los valores son equivalentes.
- Cuando usamos `===`, los valores además deben ser del mismo tipo.
 - Si el tipo de dato o el valor son diferentes devolverá falso.
 - Devolverá true cuando ambos valores son idénticos y del mismo tipo.

```
console.log(3 == "3"); // true
console.log(3 === "3"); // false
console.log(3 != "3"); // false
console.log(3 !== "3"); // true
console.log(false == null); // false (null no es equivalente a cualquier boolean).
console.log(false == undefined); // false (undefined no es equivalente a cualquier boolean).
console.log(null == undefined); // true (regla especial de JavaScript)
console.log(0 == false); // true
console.log({} >= false); // Object vacío -> false
console.log([] >= false); // Array vacío -> true
```



1.9 Estructuras de control de flujo

Si se utilizan estructuras de control de flujo, los programas dejan de ser una sucesión lineal de instrucciones para convertirse en programas inteligentes que pueden tomar decisiones en función del valor de las variables.

1.9.1 Estructura if

La estructura más utilizada en JavaScript y en la mayoría de lenguajes de programación es la estructura if. Se emplea para tomar decisiones en función de una condición. Su definición formal es:

```
if (condición) {  
    ....  
}
```

Si la condición se cumple (es decir, si su valor es true) se ejecutan todas las instrucciones que se encuentran dentro de {...}. y el programa continúa ejecutando el resto de instrucciones del script.

Ejemplo:

```
let mostrarMensaje = true;  
  
if(mostrarMensaje == true) {  
    alert("Hola Mundo");  
}
```

La comparación del ejemplo anterior suele ser el origen de muchos errores de programación, al confundir los operadores == y =. Las comparaciones siempre se realizan con el operador ==, ya que el operador = solamente asigna valores:

```
// Error - Se asigna el valor "false" a la variable  
  
if(mostrarMensaje = false) {  
    ...  
}
```

Los operadores AND y OR permiten encadenar varias condiciones simples para construir condiciones complejas:

```
var mostrado = false;
```



```
var usuarioPermiteMensajes = true;

if(!mostrado && usuarioPermiteMensajes) {
    alert("Es la primera vez que se muestra el mensaje");
}
```

1.9.2 Estructura if...else

```
if(condicion) {
    ...
}else {
    ...
}
```

Si la condición se cumple (es decir, si su valor es true) se ejecutan todas las instrucciones que se encuentran dentro del if(). Si la condición no se cumple (es decir, si su valor es false) se ejecutan todas las instrucciones contenidas en else {}.

Ejemplo

```
let price = 65;

if(price < 50) {
    console.log("Esto es barato!");
} else if (price < 100) {
    console.log("Esto no tan barato...");
} else {
    console.log("Esto es caro!");
}
```

1.9.3 Estructura switch

- Similar a otros lenguajes de programación.
- Se evalúa una variable y se ejecuta el bloque correspondiente al valor que tiene (puede ser numero, string,...).
- Al final de cada bloque pondremos la instrucción break, ya que, de no ponerlo continuaría ejecutando las instrucciones que haya en el siguiente bloque.



- Ejemplo en el que dos valores ejecutaran el mismo bloque de código:

```
let userType = 1;
switch(userType) {
  case 1:
  case 2: // Tipos 1 y 2 entran aquí
    console.log("Puedes acceder a esta zona");
    break;
  case 3:
    console.log("No tienes permisos para acceder aquí");
    break;
  default: // Ninguno de los anteriores
    console.error("Tipo de usuario erróneo!");
}
```

1.9.4 Estructura for

Crea un bucle que consiste en tres expresiones opcionales, encerradas en paréntesis y separadas por puntos y comas, seguidas de una sentencia ejecutada en un bucle.

for ([expresion-inicial]; [condicion]; [expresion-final]) sentencia

```
// ejemplo de bucle for
let limit = 5;

for (let i = 1; i <= limit; i++) { // Imprime 1 2 3 4 5
  console.log(i);
}
```

La idea del funcionamiento de un bucle for es la siguiente: "mientras la condición indicada se siga cumpliendo, repite la ejecución de las instrucciones definidas dentro del for. Además, después de cada repetición, actualiza el valor de las variables que se utilizan en la condición".

- La "inicialización" es la zona en la que se establece los valores iniciales de las variables que controlan la repetición
- La "condición" es el único elemento que decide si continua o se detiene la repetición.
- La "actualización" es el nuevo valor que se asigna después de cada repetición a las variables que controlan la repetición

```
// otro ejemplo con un array

let dias = ["Lunes", "Martes", "Miércoles", "Jueves", "Viernes", "Sábado", "Domingo"];
```



```
for(let i = 0; i < 7; i++) {  
    Console.log(dias[i]);  
}
```

1.9.5 Estructura While

La estructura while permite crear bucles que se ejecutan ninguna o más veces, dependiendo de la condición indicada:

```
let value = 1;  
  
while (value <= 5) { // Imprime 1 2 3 4 5  
    console.log(value++);  
}
```

Si la condición no se cumple ni siquiera la primera vez, el bucle no se ejecuta. Si la condición se cumple, se ejecutan las instrucciones una vez y se vuelve a comprobar la condición. Si se sigue cumpliendo la condición, se vuelve a ejecutar el bucle y así se continúa hasta que la condición no se cumpla.

Las variables que controlan la condición deben modificarse dentro del propio bucle, ya que de otra forma, la condición se cumpliría siempre y el bucle while se repetiría indefinidamente.

// El programa debe sumar todos los números menores o igual que otro dado. Por ejemplo si el número es 5, se debe calcular: 1 + 2 + 3 + 4 + 5 = 15

```
let resultado = 0;  
numero = 5;  
let i = 0;  
  
while(i <= numero) {  
    resultado += i; // es lo mismo que resultado = resultado + i;  
    i++;  
}  
alert(resultado);
```

1.9.6 estructura do...while

El bucle de tipo do...while es muy similar al bucle while, salvo que en este caso **siempre** se ejecutan las instrucciones del bucle al menos la primera vez.



```
let value = 1;

do { // Imprime 1 2 3 4 5
  console.log(value++);
} while (value <= 5);
```

De esta forma, como la condición se comprueba después de cada repetición, la primera vez siempre se ejecutan las instrucciones del bucle. Es importante no olvidar que después del `while()` se debe añadir el carácter `;` (al contrario de lo que sucede con el bucle `while simple`).

Utilizando este bucle se puede calcular fácilmente el factorial de un número: el factorial de 5 por ejemplo es $5 \times 4 \times 3 \times 2 \times 1 = 120$).

```
let resultado = 1; let numero = 5;

do {
  resultado *= numero; // resultado = resultado * numero
  numero--;
} while(numero > 0);

alert(resultado);
```

Como en cada repetición se decrementa el valor de la variable `numero` y la condición es que `numero` sea mayor que cero, en la repetición en la que `numero` valga `0`, la condición ya no se cumple y el programa se sale del bucle `do...while`.

1.9.7 Sentencias `break` y `continue`

Las sentencias `break` y `continue` permiten manipular el comportamiento normal de los bucles para detener el bucle o para saltarse algunas repeticiones. Concretamente, la sentencia `break` permite terminar de forma abrupta un bucle y la sentencia `continue` permite saltarse algunas repeticiones del bucle.

El siguiente ejemplo muestra el uso de la sentencia `break`:

```
let value = 1;

do { // Imprime 1 2
  if (value >= 3) {
    break;
  }
  console.log(value++);
} while (value <= 5);
```




Si el programa llega a una instrucción de tipo `break;`, sale inmediatamente del bucle y continúa ejecutando el resto de instrucciones que se encuentran fuera del bucle `while`.

En ocasiones, lo que se desea es saltarse alguna repetición del bucle cuando se dan algunas condiciones.

```
let value = 1;

do {                                // Imprime 1 2 4 5
    if (value == 3) {
        value++;
        continue;
    }
    console.log(value++);
} while (value <= 5);
```

En este caso, cuando `value == 3` no se termina el bucle, sino que no se ejecutan las instrucciones de esa repetición (incrementamos `value` para evitar un bucle infinito) y se pasa directamente a la siguiente repetición del bucle `while`.

Ejercicio 1.4

Crea un documento llamado **04_sumaAcumulada.js.html** y, con JavaScript, calcula y muestra la suma acumulada del 1 al 5 es decir: $1+2+3+4+5 = 15$

utilizando:

- la estructura `for(;;){...}`
- La estructura `while {...}`
- la estructura `do{...} while.`

Ejercicio 1.5

Crea un documento llamado **05_sumaAcumulada2.js.html** y, con JavaScript, pide al usuario que introduzca un número (con `prompt()`) y calcula y muestra la suma acumulada con `document.write` del 1 al número introducido por el usuario. Si el usuario introduce 4 haremos: $1+2+3+4 = 10$ utilizando el bucle: `for(;;){...}`



Ejercicio 1.6

Crea un documento llamado **06_factorial_js.html**. El factorial de un número entero n es una operación matemática que consiste en multiplicar todos los factores $n \times (n-1) \times (n-2) \times \dots \times 1$. Así, el factorial de 5 (escrito como $5!$) es igual a: $5! = 5 \times 4 \times 3 \times 2 \times 1 = 120$

Realiza un script que pida un número y calcule y muestre su factorial utilizando:

- la estructura `for(;;){...}`
- La estructura `while {...}`
- la estructura `do{...} while.`